

MCDIA500

PROGRAMACIÓN PARA LA

CIENCIA DE DATOS

AVANCE DEL PROYECTO —

FASES 1 Y 2

Nombre integrante/s:

José Ignacio Rodríguez – Yerko Guillermo Gallardo Miranda –
Sebastián Eduardo Rojas Araya

Curso: Programación para la Ciencia de Datos

Fecha: 16-09-2026

Repositorio: https://github.com/joserodriguezc/grupo_1_programacion_ciencia_datos

I. Portada e Índice

APARTADOS

I. Portada e Índice.....	2
II. Introducción y contextualización	3
III. Definición de la problemática y objetivos del proyecto	4
IV. Aplicación de herramientas científicas.....	7
V. Reproducibilidad técnica	8
VI. Control de versiones.....	9
VII. Documentación del proceso	11
VIII. Repositorio GitHub correspondiente a las Fases 1 y 2	16
IX. Codificación funcional.....	19
X. Preprocesamiento y transformación del dataset (Fase 2).....	24
XI. Validación técnica y verificación del código	29
XII. Notebook F1_Definición.ipynb (F1)	33
XIII. Notebooks ejecutables de la Fase 2.....	37
XIV. Vinculación del mapa conceptual con el avance de proyecto	40
XV. Bibliografía y aspectos formales.....	44

II. Introducción y contextualización

La protección de los datos personales y el derecho a la privacidad son materias centrales para la ética y la gobernanza de datos contemporánea. En Chile, la tramitación legislativa del proyecto de ley sobre protección de datos personales (boletín 11092-07) constituye un caso concreto y verificable para observar cómo los diputados y diputadas, y los partidos políticos a los que pertenecen, se posicionan frente a estas materias. La Cámara de Diputados y Diputadas de Chile publica los registros nominales de sus votaciones a través de un servicio de datos abiertos (Cámara de Diputados y Diputadas de Chile, s. f.), pero esa información se entrega como una relación simple entre el nombre de cada parlamentario y el sentido de su voto: no ofrece, por sí sola, indicadores de posición ideológica relativa, estabilidad individual ni cohesión partidaria.

El problema que aborda este proyecto es, por tanto, la falta de evidencia sistemática y reproducible sobre el posicionamiento ideológico y los patrones de cohesión legislativa que se manifiestan en las votaciones de este proyecto de ley específico. Transformar registros nominales dispersos en indicadores comparables exige un proceso de obtención, integración, procesamiento y análisis que sea a la vez riguroso y auditable, ya que las conclusiones que se deriven de él tendrán implicancias sobre cómo se interpreta el comportamiento de actores políticos concretos.

Existe una justificación sustantiva para este análisis: la protección de datos personales no es solo una materia técnico-legal, sino un asunto de ética pública en el que el comportamiento legislativo constituye evidencia relevante sobre cómo el sistema político chileno procesa los derechos digitales de la ciudadanía. Conocer si el posicionamiento de diputadas, diputados y partidos frente a este boletín es estable o errático, y si la cohesión interna de las bancadas se mantiene o se fragmenta, permite contrastar el discurso público sobre privacidad y datos personales con el comportamiento efectivo en la sala, e identificar coaliciones o clivajes que no son evidentes a partir de declaraciones o programas partidarios. Esta evidencia cobra particular relevancia en el contexto actual, en que la entrada en vigencia de la Ley N.º 21.719 y la creación de la Agencia de Protección de Datos Personales han situado la gobernanza de datos en el centro de la agenda pública chilena: comprender cómo se comportó el Congreso frente a esta materia antes de la nueva institucionalidad aporta una línea base empírica para el escrutinio ciudadano y académico sobre la forma en que se construyen las políticas de protección de datos en el país.

El objetivo general del proyecto es analizar el posicionamiento ideológico relativo y la estabilidad del comportamiento legislativo de diputadas y diputados, así como el posicionamiento y la cohesión de los partidos políticos, a partir de la totalidad de las votaciones nominales asociadas al boletín 11092-07. Este informe da cuenta del avance correspondiente a las Fases 1 y 2 del proyecto: la definición del problema y la configuración de un entorno reproducible (Fase 1), y la obtención, exploración, limpieza, transformación e integración inicial de los datos (Fase 2). Las fases 3 (experimentación y selección de modelos y métricas para medir el posicionamiento ideológico y la cohesión de diputados y partidos) y 4 (visualización y comunicación de resultados) se encuentran fuera del alcance de esta entrega y se documentan aquí únicamente como trabajo proyectado.

La justificación técnica de abordar esta problemática mediante programación y análisis de datos, en lugar de una revisión manual, radica en el volumen y la estructura de la evidencia: 15 votaciones, 157 diputados y 1.993 votos nominales que deben integrarse con información temporal de militancia partidaria sujeta a cambios durante el período. Solo un pipeline reproducible permite sostener, con evidencia verificable, cada decisión de tratamiento de datos que finalmente respalde un indicador de posición política.

III. Definición de la problemática y objetivos del proyecto

Formulación del problema y preguntas de análisis

La problemática que aborda este proyecto es el déficit de transparencia sobre el comportamiento legislativo real en materia de protección de datos en Chile: aunque el sentido de cada voto es público, la posición política que ese voto efectivamente expresa no lo es, porque no es factible reconstruirla de forma sencilla desde la página web de la Cámara de Diputadas y Diputados. Este nivel de transparencia exige aplicar técnicas de ingeniería y análisis de datos avanzados. Mientras eso no ocurra, la ciudadanía, la prensa y la propia academia carecen de una base verificable para determinar si el compromiso de un partido con la protección de datos personales es coherente y sostenido en el tiempo, o si su discurso público se fractura en el comportamiento efectivo de sus integrantes en la sala. No se trata, entonces, de falta de acceso al dato crudo, sino de falta de acceso a su significado político. Superar este déficit exige transformar los registros nominales dispersos de la Cámara de Diputadas y Diputados en un proceso reproducible y auditable que produzca indicadores de posición ideológica relativa, estabilidad individual y cohesión partidaria. El proyecto busca construir esos indicadores a partir de la totalidad de las votaciones nominales del boletín 11092-07¹, orientado por cuatro preguntas centrales:

- ¿Qué posiciones políticas relativas presentan las diputadas y los diputados a partir de sus patrones de votación durante la tramitación del proyecto de ley?
- ¿Qué grado de estabilidad o variabilidad presenta el comportamiento individual de las diputadas y los diputados a lo largo de las votaciones analizadas?
- ¿Qué posiciones políticas relativas presentan los partidos políticos a partir del comportamiento de sus integrantes?
- ¿Qué grado de cohesión o dispersión presentan los partidos políticos durante la tramitación del proyecto de ley?

Objetivo general

Analizar las posiciones políticas relativas y la estabilidad del comportamiento legislativo de las diputadas y los diputados, así como las posiciones políticas relativas y el grado de cohesión de los partidos políticos, a partir de todas las votaciones nominales asociadas al proyecto de ley sobre protección de datos personales en la Cámara de Diputadas y Diputados de Chile.

Objetivos específicos

- Obtener el catálogo completo y los detalles nominales de todas las votaciones asociadas al proyecto de ley.
- Identificar los períodos legislativos correspondientes a las votaciones y obtener la información de las diputadas y los diputados en ejercicio, junto con su militancia vigente en la fecha de cada votación.
- Explorar, limpiar, normalizar e integrar los datos obtenidos, diferenciando adecuadamente votos, abstenciones, ausencias y dispensas.
- Construir una tabla longitudinal (big table) y una matriz de diputadas y diputados por votación que

¹ La posición ideológica que se estima corresponde a una posición relativa dentro de esta agenda legislativa específica, y no debe leerse como una medición general de la ideología de cada diputado, diputada o partido.

cumplan los requisitos de entrada para modelos tales como B-Call, algoritmos de clustering y de indicadores complementarios.

- Estimar las posiciones políticas relativas de las diputadas y los diputados y evaluar la estabilidad de su comportamiento.
- Estimar las posiciones políticas relativas de los partidos y medir su grado de cohesión o dispersión.
- Validar la calidad de los datos y la consistencia metodológica de los resultados, documentando supuestos, decisiones y limitaciones.
- Comunicar los resultados mediante visualizaciones, documentación científica y artefactos reproducibles vinculados al repositorio del proyecto.

Alcance, exclusiones y supuestos

Tabla 1

Alcance, exclusiones y supuestos del proyecto en las fases 1 y 2

Dimensión	Definición
Unidad de observación	Voto emitido o condición registrada para un diputado en una votación nominal.
Universo	Las 15 votaciones asociadas al boletín 11092-07.
Cobertura temporal	Determinada por las fechas del catálogo: 08-05-2023 (14 votaciones) y 26-08-2024 (1 votación).
Fuente primaria	Servicios de datos abiertos de la Cámara de Diputadas y Diputados de Chile (API SOAP/REST). https://www.camara.cl/transparencia/datosAbiertos.aspx API consultadas: - retornarPeriodosLegislativos - retornarDiputadosXPeriodo - retornarVotacionesXProyectoLey - retornarVotacionDetalle
Nivel individual	Diputado o diputada por votación. Clave lógica votacion_id + diputado_id.
Nivel colectivo	Partido o militancia vigente a la fecha de cada votación.
Resultado esperado	Posición relativa, dispersión/cohesión, participación, diagnósticos y limitaciones.

Delimitación del alcance por fases

La presente entrega comprende las fases F1 y F2 del proyecto. Su alcance llega hasta la definición metodológica del estudio y la construcción de una base analítica validada. Las fases F3 y F4 se presentan como trabajo proyectado y, por tanto, sus resultados no forman parte de esta entrega.

F1: definición y planificación

En esta fase se establecieron las preguntas que orientan el análisis, los objetivos del proyecto y los criterios que delimitan el estudio. También se diseñó el flujo general de trabajo, representado en el mapa conceptual y en el plan operativo. Este último distribuye las actividades, productos y mecanismos de verificación entre las distintas etapas del proyecto.

Además, se preparó el entorno de trabajo colaborativo mediante el repositorio, la estructura de directorios, el control de versiones y la definición de dependencias. Por tanto, F1 entrega el marco metodológico y operativo necesario para desarrollar el análisis de manera trazable y reproducible.

F2: obtención, preparación e integración de los datos

Esta fase comprende la extracción programática de los antecedentes legislativos, su almacenamiento y la transformación de los registros originales en datos adecuados para el análisis. Incluye la revisión exploratoria de las fuentes, la normalización de variables, el tratamiento de inconsistencias y la integración de la información de votaciones, parlamentarios y militancias.

Un aspecto central de F2 es la asignación temporal de la militancia correspondiente a cada diputado o diputada en la fecha de la votación. También se incorporaron verificaciones sobre identificadores, duplicados, categorías de voto, fechas, cobertura y relaciones entre las tablas.

El principal producto de esta fase es una tabla analítica longitudinal, en la que cada registro representa la situación de una persona en una votación determinada. Esta tabla constituye el insumo preparado y validado para las fases posteriores.

Aspectos excluidos de la presente entrega

En esta entrega no se incluyen:

- La estimación definitiva de posiciones relativas o dimensiones latentes.
- La construcción e interpretación final de indicadores de cohesión partidaria.
- La elaboración de clasificaciones o comparaciones definitivas entre partidos o parlamentarios.
- La selección final del modelo y de sus parámetros.
- El análisis de sensibilidad de los resultados frente a distintas codificaciones.
- La interpretación política sustantiva de los patrones encontrados.
- Las visualizaciones finales destinadas a comunicar los resultados.
- La formulación de conclusiones definitivas respecto de las preguntas de investigación.

Supuestos

- Los identificadores de votación permiten recuperar un detalle nominal único.
- Las fechas permiten asignar períodos legislativos y militancias mediante reglas temporales explícitas.
- Las categorías de voto requieren normalización antes del modelamiento.
- La ausencia o la dispensa no equivalen a una abstención ni a una posición política observada.
- Las votaciones unánimes o sin varianza deben identificarse antes de construir la matriz analítica.
- Toda inferencia a nivel de partido debe distinguir entre posición central, dispersión interna y número efectivo de observaciones.
- La validez externa queda limitada a esta agenda legislativa y a su concentración temporal.

Fases proyectadas

F3: modelamiento y análisis

La fase F3 utilizará la tabla analítica para construir la matriz diputado-votación y definir la codificación utilizada por los modelos. En esta etapa se evaluará el tratamiento de abstenciones, ausencias, dispensas y votaciones sin variabilidad suficiente.

Posteriormente, se aplicarán los métodos previstos para estimar posiciones relativas y calcular indicadores de participación, estabilidad, dispersión y cohesión. También se compararán especificaciones alternativas para determinar si los resultados son identificables, estables e interpretables. La selección del modelo definitivo deberá estar respaldada por diagnósticos y análisis de sensibilidad.

F4: interpretación y comunicación de resultados

La fase F4 estará destinada a transformar los resultados del modelamiento en evidencia comprensible. Comprenderá la elaboración de tablas y visualizaciones finales, la comparación de resultados individuales y colectivos, y la interpretación de los patrones observados en relación con las preguntas de investigación.

Asimismo, esta fase incluirá la discusión de las limitaciones del análisis, la redacción de conclusiones, la preparación del informe final y la presentación de los resultados. Cualquier afirmación sobre posicionamiento, cercanía, dispersión o cohesión deberá sustentarse en los resultados obtenidos y validados durante F3.

IV. Aplicación de herramientas científicas

El ecosistema científico de Python se utiliza de forma integrada dentro de funciones con un propósito declarado. La tabla siguiente resume dónde y para qué se emplea cada herramienta a lo largo del pipeline.

Tabla 2

Herramientas científicas de Python y su uso concreto en el proyecto

Herramienta	Uso concreto en el proyecto
Python 3.12	Lenguaje de programación utilizado para configuración, extracción, transformación y validación durante las fases 1 y 2 del proyecto.
Pandas 3.0.5	Lectura/escritura de CSV y XML, perfilamiento (dtypes, nulos, duplicados), uniones controladas (merge con validate='many_to_one'/'one_to_one' e indicator=True para antiunioniones), groupby para conciliar totales oficiales con conteos nominales, y construcción de la tabla analítica final.
NumPy 2.5.3	Soporte vectorial subyacente a las operaciones de pandas y a los cálculos estadísticos de participación en el EDA.
Zeep 4.3.3 / lxml 6.1.3	Cliente SOAP para consumir los servicios retornarPeriodosLegislativos y retornarDiputadosXPeriodo, y extracción del envelope XML crudo mediante HistoryPlugin para preservar evidencia de la respuesta original.
requests + xml.etree	Consumo de los servicios REST retornarVotacionesXProyectoLey y retornarVotacionDetalle, con verificación de tipo de contenido y de validez del XML antes de aceptar la respuesta.
JupyterLab 4.6.3	Generación de notebooks con ejecución narrativa y trazable de F1 y F2.
Matplotlib 3.11.1 / Seaborn 0.13.2	Visualizaciones exploratorias del EDA de preparación: distribución de opciones de voto, histograma de participación por votación y composición por partido.
Git	Gestiona localmente las versiones y mantiene la trazabilidad de los cambios realizados.
GitHub	Aloja el repositorio y permite colaborar mediante ramas, <i>pull requests</i> y test automáticos.
uv 0.11.11	Creación del entorno, sincronización y resolución exacta mediante uv.lock
Pytest	Ejecuta pruebas automatizadas para verificar funciones, contratos de datos y ejecución de notebooks.
ruff	Revisa la calidad y el estilo del código Python, detectando errores e inconsistencias.

V. Reproducibilidad técnica

El proyecto utiliza un entorno virtual reproducible basado en Python 3.12 (Python Software Foundation, 2026) y administrado con uv (Astral, s. f.). La versión de Python se declara en `.python-version`, mientras que `pyproject.toml` establece el rango compatible `>=3.12,<3.13`. Al ejecutar `uv sync`, la herramienta crea automáticamente el ambiente virtual `.venv`, instala los componentes requeridos y evita depender de configuraciones preexistentes en el equipo del usuario.

La gestión de dependencias se realiza mediante dos archivos complementarios. `pyproject.toml` declara los paquetes requeridos por el proyecto y `uv.lock` conserva la resolución exacta y determinista de sus versiones, incluyendo las dependencias transitivas necesarias para la ejecución. El uso de `uv sync --locked --group dev` comprueba que el archivo de bloqueo esté actualizado e instala exactamente el entorno versionado, sin modificarlo durante la instalación. Por su parte, `uv.toml` utiliza `link-mode = "copy"`, lo que mejora la compatibilidad del entorno entre distintos sistemas de archivos y equipos.

Tabla 3

Dependencias declaradas en `pyproject.toml` según grupo

Grupo	Dependencias declaradas (<code>pyproject.toml</code>)
Principales	ipykernel, jupyterlab, lxml, matplotlib, numpy, pandas, seaborn, zeep
Desarrollo	nbclient, nbformat, pytest, ruff

El archivo `uv.lock` registra además el grafo completo de dependencias resueltas, incluida `requests`, utilizada por los procesos de extracción. La combinación de `.python-version`, `pyproject.toml`, `uv.lock` y `uv.toml` permite reconstruir el mismo entorno de ejecución en cualquier equipo compatible con Python 3.12 y uv.

La instalación reproducible se realiza mediante los siguientes comandos:

Figura 1

Comandos de instalación reproducible del entorno con uv

```
</> Bash

git clone https://github.com/joserodriguezc/grupo_1_programacion_ciencia_datos.git
cd grupo_1_programacion_ciencia_datos
uv sync --locked --group dev
```

Nota. Elaboración propia. Captura de los comandos de instalación ejecutados con uv.

Después de la instalación, los componentes y la ejecución integral se verifican mediante:

Figura 2

Verificación de los componentes y de la ejecución integral del proyecto

```
</> Bash

uv run python --version
uv run jupyter lab
uv run ruff check F2/src F2/test
uv run pytest -m "not slow"
uv run pytest -m slow
```

Nota. Elaboración propia. Captura de la verificación de componentes y de la ejecución integral.

La verificación técnica contempla 25 pruebas automatizadas: 11 pruebas unitarias sobre las funciones de F2/src, 13 contratos aplicados a los datos procesados y una prueba integral que ejecuta los notebooks F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb y comprueba sus indicadores de aprobación. Las 24 pruebas rápidas y el análisis de Ruff finalizan satisfactoriamente en el entorno verificado. La prueba integral también forma parte del workflow de GitHub Actions, que instala nuevamente el ambiente desde los archivos versionados y ejecuta los controles en cada *pull request* dirigido a main, lo cual se detalla en la sección XI.

Adicionalmente, F1_Definición.ipynb incorpora la función `collect_environment()`, que registra en cada ejecución la fecha UTC, la versión de Python, el ejecutable activo, el sistema operativo y las versiones de pandas, NumPy, Jupyter Core, IPython, uv y Git. Esta evidencia permite comprobar que cada componente se encuentra correctamente instalado e identificar el entorno exacto en el que se generaron los resultados.

Los scripts de F2/src resuelven sus rutas desde la ubicación del propio archivo mediante `Path(__file__).resolve().parent.parent`, mientras que los notebooks localizan automáticamente la carpeta F2 recorriendo sus directorios superiores. Por tanto, la lógica del pipeline no depende de rutas absolutas ni del nombre del usuario. Las rutas de Windows que permanecen en algunas salidas corresponden únicamente al registro de ejecuciones anteriores y demuestran que el proyecto fue utilizado en equipos distintos; no intervienen en su ejecución.

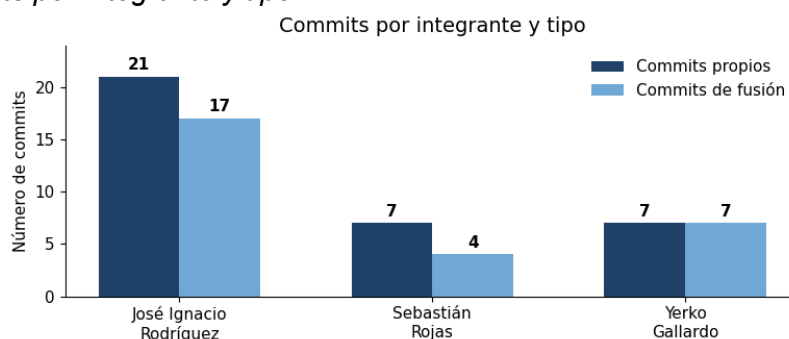
La configuración y los controles fueron revisados sobre el commit be3d083 del 15-09-2026. En conjunto, uv, el ambiente virtual y los archivos de reproducibilidad aseguran que el proyecto pueda instalarse y ejecutarse íntegramente sin depender del estado local de un equipo, garantizando estabilidad y consistencia en cualquier plataforma compatible. No se ha definido una semilla aleatoria porque F1 y F2 no contienen procedimientos estocásticos; esta deberá incorporarse en F3 cuando se ejecuten B-Call y otros métodos de modelamiento que utilicen inicialización aleatoria.

VI. Control de versiones

El equipo trabaja con Git y GitHub (GitHub, s. f.) desde el primer día del proyecto para gestionar versiones, mantener la trazabilidad de los cambios, coordinar el trabajo colaborativo y resguardar la reproducibilidad del desarrollo, con la rama main protegida y un flujo de integración basado en pull requests: toda tarea se desarrolla en una rama de corta duración con prefijo semántico (feature/, feat/, fix/, docs/, chore/, refactor/, analysis/ o ci/) y se integra mediante PR. Este criterio de versionamiento está documentado explícitamente en el propio notebook F1_Definición.ipynb.

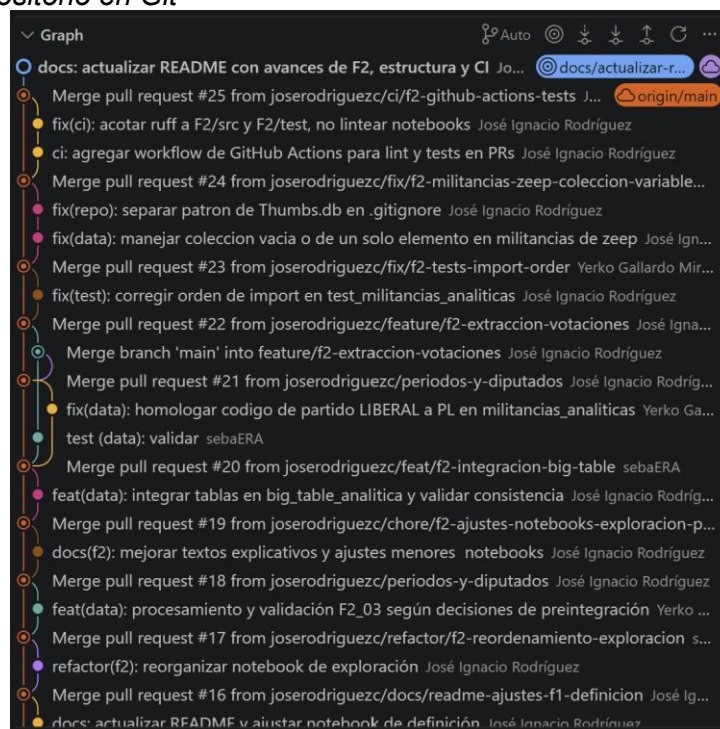
Tabla 4*Indicadores de control de versiones al cierre de la entrega*

Indicador	Evidencia al cierre de esta entrega (15-09-2026)
Commits totales	63, registrados entre el 08-09-2026 y el 15-09-2026.
Ramas de trabajo	16 ramas además de main, todas integradas por pull request.
Pull requests fusionadas	27 merges
Commits por integrante	José Ignacio Rodríguez: 38 · Sebastián Rojas: 11 · Yerko Gallardo: 14.
Convención de mensajes	Prefijos feat, fix, docs, chore, refactor, analysis y test, consistentes con <i>Conventional Commits</i> (<i>Conventional Commits</i> , s. f.).

Figura 3*Distribución de commits por integrante y tipo*

Nota. Elaboración propia a partir del historial de commits del repositorio (commit be3d083, 15-09-2026).

El número de *commits* representa la frecuencia de actividad, pero no necesariamente la magnitud o complejidad de los aportes. José concentró principalmente su trabajo en la definición del proyecto, documentación, configuración del entorno, integración de datos, control de calidad, automatización de pruebas mediante GitHub Actions y ajustes puntuales al código. Por su parte, Sebastián y Yerko agruparon parte importante de su trabajo en *commits* de mayor extensión, asociados a componentes sustantivos de extracción, exploración, procesamiento, validación y pruebas. Por ello, la contribución del equipo debe evaluarse considerando tanto el historial de cambios como la naturaleza de los componentes desarrollados.

Figura 4*Historial trazable del repositorio en Git*

Nota. Imagen ilustrativa del historial trazable del repositorio.

VII. Documentación del proceso

La presente documentación del proyecto reconstruye el proceso completo seguido en F1 y F2, desde la definición del problema y la carga de los datos hasta la obtención de una tabla analítica validada.

Proceso desde la definición hasta el resultado

Tabla 5*Proceso documentado desde la definición hasta el resultado*

Etapa	Proceso documentado	Decisión y justificación	Resultado
1. Definición	Se delimitaron el problema, las preguntas, los objetivos, la unidad de observación y el alcance.	Se definió como unidad analítica el voto o condición registrada para un diputado en una votación nominal. Esto permite analizar posteriormente comportamiento individual y agregación partidaria.	Alcance metodológico, mapa conceptual y plan operativo.
2. Obtención	Los scripts 01 a 04 consultaron los servicios de la Cámara para	Las respuestas originales se conservaron como XML para disponer de evidencia verificable y evitar que cambios posteriores en	17 XML: proyecto, diputados y 15 votaciones.

	recuperar el proyecto, los períodos, los diputados, las militancias y el detalle nominal.	la API impidan reconstruir la extracción.	
3. Estructuración	Los XML se transformaron en cinco CSV intermedios.	Los datos intermedios se separaron de los originales para evitar sobrescribir la fuente y para reutilizarlos sin efectuar nuevas solicitudes al servicio.	Proyecto, períodos, diputados, militancias y detalle de votaciones.
4. Exploración	Se examinaron dimensiones, tipos, nulos, duplicados, claves, categorías, integridad referencial, temporalidad, participación y unanimidad preliminar.	F2_02_exploracion.ipynb descubre automáticamente los CSV para inventariarlos, pero identifica por estructura las cinco tablas fuente utilizadas en los controles. Esto evita depender exclusivamente del nombre de los archivos.	Hallazgos de calidad y matriz de 16 controles exploratorios.
5. Procesamiento	Se normalizaron identificadores, fechas, totales, categorías y columnas.	Los identificadores se conservaron como texto; las fechas no convertibles y los totales no numéricos detienen el proceso; los nulos válidos se mantienen sin imputación. Estas reglas evitan alterar artificialmente la información de origen.	Cinco tablas normalizadas y un reporte estructurado de calidad.
6. Resolución temporal	Se construyó una línea de tiempo analítica de militancias sin solapamientos.	Se mantuvieron dos representaciones: militancias_observadas, que conserva lo entregado por la fuente, y militancias_analiticas, que contiene las reglas necesarias para integrar los votos. Así se diferencia evidencia de decisión analítica.	407 registros observados y 399 registros analíticos.
7. Integración	El detalle nominal se vinculó con el proyecto, los diputados y la militancia vigente en	El detalle de votaciones se utilizó como tabla vertebral. Los cruces fueron diseñados para conservar exactamente su cardinalidad y evitar la pérdida o duplicación de	big_table_analitica.csv, con 1.993 filas y 34 columnas.

	la fecha de cada voto.	votos.	
8. Validación	Se comprobaron claves, cardinalidad, cobertura, consistencia, temporalidad, categorías y casos particulares.	La integración solo se aprueba cuando las 12 validaciones presentan estado OK. El veredicto se calcula durante la ejecución y no corresponde a un texto escrito manualmente.	Integración aprobada, diagnóstico final y 25 pruebas automatizadas.
9. Resultado de F2	Se consolidó una base longitudinal preparada para el modelamiento.	F2 termina en la disponibilidad de datos estructurados y validados. Las posiciones relativas y los indicadores de cohesión no se calculan todavía, porque corresponden a F3 y F4.	Insumo reproducible para experimentación y modelamiento.

Decisiones técnicas adoptadas y su justificación

El procesamiento fue guiado por el principio de no corregir automáticamente una anomalía sin evidencia suficiente. Cuando un dato presentaba problemas, se evaluó si debía conservarse, excluirse, homologarse o reemplazarse por otra fuente más confiable.

Tabla 6

Decisiones de procesamiento, justificación e impacto

Decisión	Justificación	Impacto
No modificar los datos intermedios	Los CSV de data/interim representan la estructuración directa de la fuente.	Permite comparar los resultados procesados con los datos originales y reproducir las transformaciones.
No imputar nulos válidos	Algunos campos vacíos responden a la estructura de la API y no a pérdida aleatoria de información.	Se evita introducir valores que no fueron observados.
Eliminar campos sin cobertura	rut, rut_dv y nombre2 no contienen información utilizable en las tablas correspondientes.	Reduce columnas sin valor analítico sin afectar la identificación de los diputados.
No utilizar las fechas de período contenidas en diputados	En 155 de 157 diputados presentan una estructura temporal que no coincide con la semántica esperada para la fecha de inicio del período legislativo 2022-2026.	La vigencia se determina mediante periodos.csv, mientras la anomalía se conserva en el reporte de calidad.
Conservar diputados sin	La ausencia de participación es un resultado observado y no debe eliminar	Se mantienen seis diputados sin votos, pero no se crean registros artificiales en

votos	al diputado del dato maestro.	la tabla de detalle de votaciones.
Homologar LIBERAL a PL	Ambos códigos representan al Partido Liberal de Chile.	Se evita tratar un mismo partido como dos categorías distintas; cinco registros fueron homologados.
Mantener una fila por diputado y votación	Esta es la granularidad definida para el análisis.	La clave <code>votacion_id + diputado_id</code> permanece única durante el procesamiento y la integración.
Usar el detalle como tabla vertebral	Cada fila del detalle de votaciones representa una observación analítica que debe preservarse.	La tabla final mantiene las 1.993 filas originales.
No duplicar atributos equivalentes	Proyecto y detalle contienen variables semánticamente iguales con nombres diferentes.	Se verificó su consistencia y solo se incorporaron los atributos nuevos, evitando columnas redundantes y sufijos <code>_x/_y</code> .
Aplicar una integración temporal cerrada	La militancia debe estar vigente el mismo día de la votación.	Cada voto recibe exactamente una militancia mediante $\text{fecha_inicio} \leq \text{fecha_votacion} \leq \text{fecha_termino}$; los intervalos abiertos se admiten cuando no existe fecha de término.

Decisiones particulares sobre militancias

Los casos de Carter, Bugueño y Veloso no se resolvieron mediante una regla automática porque presentaban registros superpuestos o secuencias temporales respecto de sus militancias cuya aplicación literal generaba resultados ambiguos. Las decisiones se documentaron en `F2_03_procesamiento_validacion.ipynb`, se incorporaron a `reporte_calidad.csv` y posteriormente se comprobaron de manera explícita en `F2_04_Integración.ipynb`.

Tabla 7

Casos resueltos mediante revisión manual de militancias

Diputado	Decisión y fundamento	Impacto en la integración
Carter (id 1017)	Para las fechas analizadas se conserva UDI. Se excluyen registros redundantes de UDI e Independiente que se superponían con la línea temporal seleccionada.	Las votaciones quedan vinculadas a una única militancia (UDI) y se eliminan coincidencias múltiples.
Bugueño (id 1114)	Se extiende FRVS hasta el término del período relevante. La decisión se basa en que 14 de las 15 votaciones coinciden con FRVS y la votación restante queda en el tramo ambiguo registrado sucesivamente como Independiente y Frente Amplio.	Las 15 votaciones reciben una afiliación única (FRVS) y se evita una ruptura temporal que afectaría solo a la última observación con el objetivo de mantener la coherencia analítica.

Veloso (id 1180)	Se extiende RD hasta el término del período, porque 14 de las 15 votaciones coinciden con RD y solo una aparece en el tramo registrado como Independiente.	Las votaciones quedan asociadas a una línea temporal única y sin solapamientos. Se selecciona RD para mantener la coherencia analítica.
-------------------------	--	---

Estas decisiones no modifican los datos observados: los registros originales permanecen en `militancias_observadas.csv`. Su efecto se limita a la tabla analítica empleada en la integración y debe considerarse posteriormente como una limitación metodológica.

Registro de cambios significativos

Las modificaciones principales fueron registradas mediante Git y mensajes semánticos en los commits. Los cambios no solo identifican qué se modificó, sino también el motivo y su impacto sobre las etapas posteriores.

Tabla 8

Cambios principales registrados en el historial de versiones

Cambio	Motivo	Impacto en el desarrollo
Orquestación de la obtención (021cd96)	Reunir los procesos de extracción en una secuencia reproducible.	Permitió generar ordenadamente los datos originales e intermedios requeridos por el EDA de pre-procesamiento.
Procesamiento y validación (6550e3b)	Implementar las reglas derivadas de los hallazgos exploratorios.	Transformó los datos intermedios en seis productos procesados y documentó las decisiones en el reporte de calidad.
Integración analítica (d380927)	Construir una tabla con la granularidad diputado-votación.	Produjo la tabla de 1.993 filas utilizada como entrada de las fases posteriores.
Homologación LIBERAL → PL (1b068e8)	Corregir la coexistencia de dos códigos para un mismo partido.	Normalizó cinco registros y actualizó el reporte de calidad de 168 a 173 observaciones.
Pruebas automatizadas (30e4059)	Convertir las validaciones de notebooks en controles reutilizables.	Incorporó pruebas unitarias, contratos de datos y una prueba de ejecución integral.
Integración continua (1e97d4a, e9de084)	Detectar errores antes de integrar modificaciones en main.	Automatizó Ruff, las pruebas rápidas y la ejecución de notebooks en cada PR.
Ejecución determinista (7421258, 1e79764)	Evitar diferencias de orden entre ejecuciones y conservar el indicador de término.	Estabilizó <code>F2_03_procesamiento_validacion.ipynb</code> y permitió verificar automáticamente su ejecución.
Actualización del	Corregir la diferencia	El README pasó a reflejar el procesamiento, la

README (a8f5a12)	entre la documentación y el estado efectivo de F2.	integración, las pruebas y la CI vigentes.
-------------------------	--	--

VIII. Repositorio GitHub correspondiente a las Fases 1 y 2

El proyecto se encuentra disponible en el repositorio público [grupo 1 programacion ciencia datos](https://github.com/joserodriguezc/grupo_1_programacion_ciencia_datos). A continuación, se presenta evidencia para cada elemento respaldado en el mismo.

Inclusión de las carpetas F1 y F2

El repositorio se organiza por fases y separa la definición del proyecto de las actividades de obtención, procesamiento e integración de datos.

Tabla 9

Estructura del repositorio: rutas, contenido y función

Ruta	Contenido y función
F1/notebooks/	Contiene F1_Definición.ipynb, con la problemática, preguntas, objetivos, alcance, supuestos, entorno y planificación.
F1/docs/	Contiene el informe, el mapa conceptual, el plan operativo y los documentos de referencia.
F2/notebooks/	Contiene los cuatro notebooks que documentan la obtención, exploración, procesamiento-validación e integración.
F2/src/	Contiene los cuatro scripts de extracción y el módulo reutilizable validaciones.py.
F2/data/raw/	Conserva 17 XML originales: proyecto, diputados y 15 votaciones.
F2/data/interim/	Contiene cinco CSV intermedios estructurados a partir de los XML.
F2/data/processed/	Contiene ocho CSV procesados, incluidos big_table_analitica.csv, reporte_calidad.csv y diagnostico_integracion.csv.
F2/test/	Contiene pruebas unitarias, contratos de datos y una prueba integral de ejecución de notebooks.

La separación entre F1 y F2 permite vincular los artefactos con la fase que los produjo y reconstruir el desarrollo desde la definición del problema hasta la obtención de la tabla analítica.

README estructurado

El README.md, actualizado en el commit a8f5a12 del 15-09-2026 y disponible en https://github.com/joserodriguezc/grupo_1_programacion_ciencia_datos/blob/main/README.md, contiene:

- Propósito y tema del proyecto.
- Pregunta principal y objetivo general.
- Identificación de los integrantes.

- Estado de avance de F1 a F4.
- Descripción detallada de los productos de F2.
- Estructura actual del repositorio.
- Requisitos de Python, uv y Git.
- Instrucciones de instalación y ejecución.
- Comandos para ejecutar JupyterLab, Pytest y Ruff.
- Identificación de la fuente de datos.
- Descripción de las herramientas principales.
- Estado de las pruebas y de la integración continua.

El documento registra F1 como completa y F2 con la extracción, exploración, procesamiento, validación e integración finalizadas. F3 y F4 permanecen identificadas como etapas pendientes, evitando presentar como terminados componentes que todavía no han sido desarrollados.

Historial completo de commits

Al cierre de la revisión, la rama main se encuentra en el commit be3d083. El historial conserva **63 commits**, registrados entre el 08-09-2026 y el 15-09-2026, estando disponible en :

Tabla 10

Distribución de los commits según tipo

Tipo de commit	Cantidad
Commits propios o de contenido	35
Commits de fusión	28
Total	63

De los 28 commits de fusión, 27 corresponden a pull requests. El commit adicional corresponde a una fusión de main dentro de una rama de trabajo.

El historial completo permanece disponible en https://github.com/joserodriguezc/grupo_1_programacion_ciencia_datos/commits/main/ y permite revisar, para cada modificación:

- Autor y fecha.
- Mensaje del commit.
- Archivos agregados o modificados.
- Diferencias respecto de la versión anterior.
- Rama de origen.

- Pull request mediante el cual se integró.

El repositorio conserva la secuencia completa de decisiones, implementaciones, correcciones e integraciones realizadas durante el proyecto.

Control de versiones alineado con el desarrollo

El control de versiones refleja la evolución efectiva de F1 y F2. Después de la configuración inicial, el trabajo se realizó principalmente en ramas de corta duración y se incorporó a main mediante pull requests.

Se crearon 16 ramas remotas de trabajo, además de main, con prefijos como feature/, feat/, fix/, docs/, chore/, refactor/ y ci/.. Los mensajes utilizan principalmente feat, fix, docs, chore, refactor, analysis, test y ci, siguiendo *Conventional Commits*.

Tabla 11

Desarrollo registrado por período y evidencia principal

Período	Desarrollo registrado	Evidencia principal
08-09-2026	Configuración inicial del entorno reproducible y registro de integrantes.	fc99d92, 8896487, 211e8bf.
12-09-2026	Implementación de F1, incorporación de documentación e inicio de los procesos de extracción.	0c7c64f, 014f783, f5abda4, 76eaf72.
13-09-2026	Orquestación de la obtención, extracción del detalle y desarrollo del EDA.	887cc2f, 021cd96, 133f685, 984c4dc.
14-09-2026	Procesamiento, validación, integración de la tabla analítica y pruebas automatizadas.	6550e3b, d380927, 30e4059.
15-09-2026	Integración continua, actualización del README y ajustes de estabilidad.	1e97d4a, e9de084, a8f5a12, 1e79764.

Esta correspondencia demuestra que los commits se incorporaron en relación con un hito concreto del flujo de trabajo.

Además, .github/workflows/ci.yml ejecuta Ruff, las pruebas rápidas y la prueba integral de notebooks en los pull requests dirigidos a main. De este modo, el control de versiones se conecta con la verificación técnica del proyecto y permite detectar errores antes de incorporar nuevos cambios.

5. Contribuciones individuales identificables y trazables

Las contribuciones pueden atribuirse a cada integrante mediante los datos de autor, los mensajes de commit, los archivos modificados y los pull requests asociados.

Tabla 12

Contribuciones por integrante según commits y aportes

Integrante	Commits propios	Commits de fusión	Total	Commits y aportes principales
José Ignacio	21	17	38	0c7c64f: definición de F1; d380927: integración de la tabla analítica; 1e97d4a: GitHub Actions; a8f5a12:

Rodríguez				actualización del README.
Sebastián Rojas (sebaERA)	7	4	11	76eaf72: extracción de votaciones; 887cc2f: detalle nominal; 133f685 y 984c4dc: exploración; 30e4059: pruebas automatizadas.
Yerko Gallardo	7	7	14	f5abda4: extracción de diputados; 021cd96: orquestación de la obtención; 6550e3b: procesamiento y validación; 1b068e8: homologación de partidos; 1e79764: estabilidad de F2_03_procesamiento_validacion.ipynb.
Total	35	28	63	Historial completo del equipo.

IX. Codificación funcional

La codificación separa la orquestación del flujo y la implementación de las tareas. F2_01_obtencion.ipynb controla el orden operacional, mientras que los módulos de F2/src encapsulan la consulta, transformación, validación y escritura de los datos en funciones con responsabilidades y parámetros definidos.

Esta separación permite ejecutar y probar cada componente de manera independiente, reutilizar funciones sin copiar código entre notebooks y verificar que la salida de una etapa sea compatible con la entrada de la siguiente.

Flujo operacional

El pipeline de obtención sigue una secuencia explícita:

1. Recuperar el catálogo de votaciones del boletín.
2. Obtener los períodos legislativos disponibles.
3. Relacionar las fechas de las votaciones con el período correspondiente.
4. Recuperar los diputados y las militancias del período identificado.
5. Descargar y consolidar el detalle nominal de las votaciones.

Tabla 13

Etapas del flujo de procesamiento: entradas, salidas y controles

Etapa	Función principal	Entrada	Salida	Control de coherencia
01. Catálogo	extraer_votaciones(numero_boletin)	Boletín 11092-07	proyecto_ley.xml, proyecto_ley.csv y un DataFrame	Se obtienen 15 votaciones y sus identificadores.

02. Períodos	extraer_periodos()	Servicio retornarPeriodosLegislativos	periodos.csv y un DataFrame ordenado temporalmente	Se recuperan 11 períodos con fechas de inicio y término.
03. Selección temporal	buscar_periodo(fecha_votacion, df_periodos)	Fecha de cada votación y tabla de períodos	periodo_id asociado a cada votación	Las 15 votaciones se asignan al período legislativo 10.
04. Diputados	extraer_diputados(periodo_id)	periodo_id = 10	XML SOAP, diputados.csv y militancias.csv	Se generan 157 diputados y 407 registros de militancia.
05. Detalle nominal	extraer_detalle_votaciones()	Identificadores contenidos en proyecto_ley.csv	15 XML y detalle_votaciones.csv	Se consolidan 1.993 filas con clave lógica votacion_id + diputado_id.

La dependencia entre las etapas es verificable: el catálogo generado por el script 01 proporciona las fechas utilizadas para seleccionar el período y los identificadores consumidos por el script 04; el catálogo de períodos generado por el script 02 determina el parámetro utilizado por el script 03; y los resultados de los cuatro módulos constituyen las cinco tablas de entrada de F2_02_exploracion.ipynb y F2_03_procesamiento_validacion.ipynb.

Funciones de obtención y transformación

Tabla 14

Funciones de obtención y transformación de datos

Función	Parámetros y retorno	Responsabilidad
extraer_votaciones(numero_boletin: str)	Recibe un boletín y retorna DataFrame.	Consulta el catálogo, conserva el XML original y genera el

		CSV de votaciones.
construir_filas_periodos(periodos)	Recibe la colección SOAP y retorna una lista de registros.	Transforma los objetos del servicio en filas estructuradas.
extraer_periodos()	Sin parámetros; retorna DataFrame.	Consulta, ordena y guarda el catálogo de períodos legislativos.
construir_filas_diputados(diputados_perodo, periodo_id)	Recibe una colección y el período.	Genera una fila normalizada por diputado.
construir_filas_militancias(diputados_perodo)	Recibe la colección de diputados del período.	Genera el historial de militancias y controla colecciones vacías, unitarias o múltiples.
escribir_csv(filas, columnas, ruta_archivo)	Recibe registros, esquema y ruta.	Centraliza la creación de directorios y escritura de CSV.
extraer_diputados(periodo_id)	Recibe el período legislativo.	Consulta el servicio, conserva el envelope SOAP y genera las tablas de diputados y militancias.
obtener_ids_votaciones(archivo_csv=VOTACIONES_CSV)	Recibe la ruta del catálogo y retorna list[int].	Verifica el archivo y la columna Id, elimina nulos y duplicados y entrega los identificadores.
descargar_xml_votaciones(ids_votaciones)	Recibe una lista de identificadores y retorna rutas.	Descarga y valida un XML por votación.
leer_xml_votacion(archivo_xml)	Recibe una ruta y retorna DataFrame.	Convierte un XML en registros nominales con metadatos de la votación.

consolidar_xml_votaciones(archivos_xml)	Recibe rutas y retorna DataFrame.	Concatena los detalles nominales sin alterar su granularidad.
guardar_votos_csv(df, archivo_salida)	Recibe un DataFrame y una ruta.	Guarda el resultado consolidado con un esquema único.
extraer_detalle_votaciones()	Sin parámetros; retorna DataFrame.	Coordina lectura de identificadores, descarga, consolidación y escritura.

Las funciones que consultan servicios se encuentran separadas de aquellas que transforman objetos o escriben archivos. Esto permite probar las transformaciones mediante datos simulados sin realizar solicitudes a la red.

Funciones de procesamiento, integración y validación

F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb incorporan funciones reutilizables para aplicar reglas homogéneas sobre las distintas tablas.

Tabla 15

Funciones reutilizables de los notebooks F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb

Función	Parámetros principales	Responsabilidad
normalizar_identificador(serie, nombre_columna)	Serie y nombre de la variable.	Convierte identificadores a texto, elimina espacios y rechaza valores vacíos.
normalizar_fecha(serie, nombre_columna, permitir_nulos=True)	Serie, nombre y regla de nulos.	Convierte fechas y detiene el proceso ante valores no interpretables.
normalizar_total(serie, nombre_columna)	Serie y nombre.	Convierte totales a valores enteros y rechaza datos no numéricos.
registrar_observacion(tabla, diputado_id, tipo, descripcion, regla_aplicada)	Contexto completo del hallazgo.	Agrega una observación trazable al reporte de calidad.
asignar_militancia_observada(diputado_id, tabla_militancias)	Diputado y tabla de militancias.	Contrasta las fechas de sus votos con los intervalos observados de militancia.

anti_join(izquierda, derecha, clave_izquierda, clave_derecha, nombre_control)	Tablas, claves y nombre del control.	Detecta registros sin correspondencia mediante una anti-uni3n.
verificar_partido_unico_esperado(diputado_id, alias_esperado, nombre_caso)	Diputado, partido y caso.	Comprueba las decisiones adoptadas para Carter, Bugue1o y Veloso.
agregar_validacion(nombre, resultado, estado, detalle)	Resultado completo de una comprobaci3n.	Incorpora el control a la matriz consolidada de validaciones.
buscar_duplicados_clave(df, columnas)	DataFrame y clave esperada.	Devuelve los registros que duplican una clave simple o compuesta.
detectar_solapamientos_vigencia(df, ...)	DataFrame y de nombres de columnas temporales.	Detecta intervalos superpuestos por diputado.
validar_columnas_exactas(df, columnas_esperadas)	DataFrame y esquema.	Verifica presencia, ausencia y orden de las columnas.
resumir_completitud(df)	DataFrame.	Calcula nulos y porcentaje de completitud por variable.

Estas funciones reciben expl3citamente los datos y par3metros que necesitan, en vez de depender de variables ocultas o de bloques de c3digo duplicados. Las reglas de normalizaci3n pueden aplicarse a varias columnas y tablas con el mismo comportamiento, manteniendo consistencia entre las entradas y las salidas.

Controles de entradas y salidas

La coherencia del flujo se asegura mediante controles incorporados en cada etapa:

- Las rutas se crean antes de escribir los archivos.
- Se comprueba la existencia de los archivos de entrada y de las columnas obligatorias.
- Los identificadores de votaci3n se convierten a enteros, se eliminan nulos y se deduplican.
- Las solicitudes HTTP utilizan un timeout de 30 segundos y `raise_for_status()`.
- El script de detalle rechaza respuestas HTML de mantenimiento aunque el servidor responda con estado HTTP 200.
- El contenido se analiza como XML antes de guardarse y procesarse.
- La granularidad se verifica mediante la clave `votacion_id + diputado_id`.

- Las integraciones comprueban que no se pierdan ni dupliquen las 1.993 observaciones.
- Las anti-uniones verifican cobertura entre proyecto, diputados, militancias y votos.
- Las transformaciones trabajan sobre copias cuando es necesario preservar los DataFrames de entrada.
- Las salidas procesadas se contrastan mediante contratos de datos y matrices de validación.

La suite automatizada verifica las funciones puras de los cuatro scripts, los casos límite de militancias, los esquemas de salida, las claves, los intervalos temporales y los contratos de las tablas procesadas. De esta manera, el flujo no depende únicamente de una ejecución visual de los notebooks: las relaciones entre entrada, procesamiento y salida pueden comprobarse de forma repetible con pytest.

En consecuencia, la codificación funcional presenta responsabilidades delimitadas, parámetros explícitos, un orden operacional verificable y controles que conectan cada entrada con su resultado. Esto permite localizar errores por etapa, reutilizar las funciones y mantener consistencia entre extracción, procesamiento, integración y validación.

X. Preprocesamiento y transformación del dataset (Fase 2)

El preprocesamiento se implementó como un pipeline reproducible compuesto por diagnóstico, normalización, resolución de problemas de calidad, integración temporal y validación final. El criterio rector fue mantener separados el dato observado y las decisiones analíticas: los archivos de data/interim permanecen intactos, mientras que todas las transformaciones se generan en data/processed.

Este diseño permite reconstruir cada resultado desde su fuente, comparar los datos antes y después de las transformaciones y modificar una regla sin perder la evidencia original.

Diagnóstico previo

F2_02_exploracion.ipynb examina las cinco tablas de entrada producidas por F2_01_obtencion.ipynb: proyecto_ley.csv, periodos.csv, diputados.csv, militancias.csv y detalle_votaciones.csv. En conjunto contienen 2.583 filas, correspondientes a unidades de observación diferentes.

Antes de transformar los datos se revisaron dimensiones, tipos, valores faltantes, duplicados, cardinalidades, claves lógicas, integridad referencial, consistencia temporal y equivalencia entre códigos y etiquetas. También se conciliaron los registros nominales con los totales oficiales y se analizaron participación, unanimidad preliminar y composición política.

Tabla 16

Hallazgos del análisis exploratorio y sus implicancias

Hallazgo	Magnitud	Implicancia para el procesamiento
Valores faltantes	2.622	Se concentran en campos estructuralmente vacíos o no aplicables; requieren clasificación y no imputación automática.
Duplicados exactos	0	No fue necesario eliminar filas por duplicación completa.
Duplicados en claves lógicas	0	Las unidades de observación se encuentran correctamente identificadas.

Espacios laterales	10 registros en Artículo	Se requiere normalización de texto antes de comparar o integrar.
Errores referenciales	0 en cinco controles	Las entidades utilizadas por el detalle existen en sus tablas maestras.
Conciliación de votos	15 de 15 votaciones correctas	Los 1.993 registros nominales concilian con TotalSi, TotalNo, TotalAbstencion y TotalDispensado.
Asignación de militancia	1.965 votos con asignación única y 28 ambiguos	Las ambigüedades deben resolverse antes de incorporar el partido a la tabla analítica.
Catálogo de partidos	30 códigos para 29 nombres	LIBERAL y PL representan al mismo partido y requieren homologación.
Matriz exploratoria	14 OK, 2 ADVERTENCIA, 0 ERROR	Las advertencias corresponden a militancias ambiguas y códigos partidarios no homologados.

No se detectaron duplicados sobre las claves esperadas: Id en proyecto, periodo_id en períodos, diputado_id en diputados, votacion_id + diputado_id en el detalle y diputado_id + partido_id + fecha_inicio en militancias.

Manejo de valores faltantes

Los valores NA no se trataron mediante una regla general de eliminación o imputación. Cada caso fue clasificado según su significado:

Tabla 17

Tratamiento de los valores faltantes según su significado

Campo o situación	Tratamiento	Justificación técnica
nombre2, rut y rut_dv	Se eliminan de diputados_procesados.csv; nombre2 también se elimina del detalle_votaciones.csv.	Presentan 0 % de cobertura y no aportan información utilizable. Imputarlos introduciría datos inexistentes.
fecha_termino_periodo	Se conserva como nulo válido.	El valor vacío representa un período vigente y no una pérdida aleatoria de información.
Articulo	Se conserva nulo para una de las 15 votaciones.	La ausencia proviene de la fuente. Después de la integración afecta 122 filas, pero no impide identificar la votación.
Identificadores obligatorios	No se permiten nulos ni cadenas vacías.	Son necesarios para las claves, relaciones e integridad del dataset.
Fechas utilizadas en reglas temporales	Los valores no convertibles detienen el pipeline.	Una fecha inválida impediría determinar período o militancia vigente.
Partido analítico	No se permiten valores vacíos en el resultado final.	Cada voto debe quedar asociado exactamente a una categoría partidaria.
Diputados sin votos	Se conservan en la dimensión de diputados, sin crear votos artificiales.	La ausencia de participación no justifica eliminar al diputado ni inventar observaciones.

La tabla final conserva únicamente dos columnas con valores faltantes: fecha_termino_periodo, con 1.993

nulos estructurales, y articulo, con 122 nulos provenientes de una votación. Ninguna clave, voto, diputado o militancia analítica queda incompleta.

Casting, normalización y transformación de variables

F2_03_procesamiento_validacion.ipynb implementa 21 decisiones de procesamiento mediante funciones reutilizables. Las transformaciones se justifican por su efecto sobre las claves, comparaciones, reglas temporales y análisis posteriores.

Tabla 18

Transformaciones aplicadas y su justificación

Transformación	Aplicación	Justificación e impacto
Casting de identificadores	diputado_id, votacion_id, partido_id y periodo_id se convierten a texto y se eliminan espacios laterales.	Los identificadores representan categorías, no magnitudes. El tratamiento como texto evita operaciones aritméticas y asegura claves consistentes.
Casting de totales	total_si, total_no, total_abstencion y total_dispensado se convierten a enteros con soporte de nulos mediante Int64.	Preserva el significado discreto de los conteos y permite validar su conciliación sin convertirlos a valores continuos.
Conversión de fechas	Las columnas temporales se convierten a datetime.	Permite ordenar observaciones, determinar períodos y evaluar intervalos de militancia. Los errores no se convierten silenciosamente en nulos.
Normalización de texto	Se eliminan espacios laterales y se normalizan textos para comparar descripciones equivalentes.	Evita interpretar diferencias de formato como discrepancias sustantivas.
Homologación de partidos	LIBERAL se transforma en PL dentro de militancias_analiticas.csv.	Ambos códigos corresponden al Partido Liberal de Chile. La regla evita generar dos categorías para una misma organización.
Voto nominal	opcion_voto conserva las categorías Afirmativo, En Contra y Abstención.	No se impone un orden ni una distancia numérica antes de conocer los requerimientos del modelo de F3.
Eliminación de columnas vacías	Se excluyen nombre2, rut y rut_dv de las tablas analíticas.	Reduce variables sin información sin afectar la unidad de observación ni las claves.
Ordenamiento final	La tabla se ordena por fecha, votación y diputado.	Genera una salida determinista y estable entre ejecuciones.

Decisión sobre escalamiento

No se aplicó escalamiento numérico en F2 porque las variables cuantitativas disponibles corresponden principalmente a conteos oficiales utilizados para control —totales de votos— y no a características continuas definidas para el modelamiento.

Escalar estos campos en esta etapa reduciría su interpretabilidad y anticiparía una decisión que depende del algoritmo seleccionado. El voto permanece como variable nominal y los identificadores como categorías.

Tratamiento de anomalías temporales

Se detectó que 155 de 157 diputados presentan `fecha_inicio_periodo` igual al inicio del último año del período legislativo 10 y `fecha_termino_periodo` nula. Como esta estructura no coincide con la semántica esperada, el equipo decidió no corregir ni reinterpretar esos valores. La vigencia legislativa se determina mediante el archivo `periodos.csv`, mientras la anomalía se conserva en `reporte_calidad.csv`.

Para las militancias se mantuvieron dos tablas:

- `militancias_observadas.csv`, con los 407 registros normalizados tal como provienen de la fuente.
- `militancias_analiticas.csv`, con 399 registros utilizables, sin solapamientos y con las decisiones temporales aplicadas.

Tabla 19

Casos temporales conflictivos y reglas aplicadas

Caso	Problema	Regla aplicada e impacto
Intervalo inválido	Una militancia presenta término anterior al inicio.	Se conserva en la tabla observada y se excluye de la línea analítica.
Carter	Registros UDI e Independiente se superponen.	Se conserva UDI para las fechas de las votaciones y se evita una asignación múltiple.
Bugueño	Las votaciones se distribuyen entre FRVS y un tramo ambiguo Independiente/Frente Amplio.	Se extiende FRVS hasta el término del período relevante, dejando una militancia única para los 15 votos.
Veloso	Catorce votos coinciden con RD y uno con un tramo registrado como Independiente.	Se extiende RD hasta el término del período, evitando una ruptura analítica basada en una sola observación.
Partido Liberal	La fuente emplea LIBERAL y PL para un mismo partido.	Se homologan cinco registros a PL, manteniendo intacta la tabla observada.

Estas decisiones quedan registradas en `reporte_calidad.csv`, que contiene 173 observaciones: 155 anomalías temporales, 7 excepciones particulares, 6 diputados sin votos y 5 homologaciones partidarias.

Integración temporal

`F2_04_Integración.ipynb` utiliza `detalle_votaciones_procesado` como tabla vertebral. La integración debe conservar las 1.993 observaciones originales: una fila por combinación `votacion_id` + `diputado_id`.

El proceso incorpora:

1. Los atributos nuevos del proyecto mediante un cruce muchos-a-uno.
2. Los datos adicionales de diputados mediante un segundo cruce muchos-a-uno.
3. La militancia vigente mediante un cruce temporal.

Antes de incorporar columnas redundantes, se comprueba que los valores del proyecto y del diputado coincidan con los existentes en el detalle. Solo se agregan los atributos nuevos, evitando duplicados y sufijos `_x/_y`.

El pipeline se detiene si un voto queda sin militancia, si coincide con más de una militancia o si el partido analítico queda vacío.

La militancia se asigna mediante un intervalo cerrado cuando existe fecha de término:

$$\text{fecha_inicio} \leq \text{fecha_votacion} \leq \text{fecha_termino}$$

Cuando la fecha de término es nula, se utiliza un intervalo abierto hacia el futuro:

$$\text{fecha_inicio} \leq \text{fecha_votacion}$$

Validación integral del resultado

La integración final se somete a 12 comprobaciones simultáneas:

- Igualdad entre la cantidad de filas del detalle y de la tabla final.
- Unicidad de votacion_id + diputado_id.
- Unicidad del identificador temporal de control.
- Ausencia de columnas duplicadas.
- Ausencia de sufijos accidentales _x y _y.
- Cobertura completa de proyecto.
- Cobertura completa de diputados.
- Cobertura completa de partido analítico.
- Una militancia vigente por fecha de voto.
- Consistencia entre metadatos y totales.
- Preservación de las columnas originales del detalle.
- Ausencia de nuevas categorías de voto.

Las 12 validaciones finalizan en estado OK. Adicionalmente, los contratos automatizados comprueban cardinalidad, claves, completitud de variables críticas, intervalos temporales, ausencia de solapamientos y permanencia de las excepciones documentadas.

Dataset final

Tabla 20

Métricas del dataset final

Métrica	Resultado
Filas	1.993
Columnas	34
Votaciones únicas	15
Diputados con voto	151
Partidos analíticos	23
Categorías de voto	3
Duplicados votacion_id + diputado_id	0
Votos sin proyecto	0
Votos sin diputado	0
Votos sin partido analítico	0
Votos con militancia ambigua	0
Rango temporal	08-05-2023 19:05:22 a 26-08-2024 19:03:25

El resultado se exporta como `big_table_analitica.csv`, con 1.993 filas y 34 columnas. La tabla preserva la granularidad original, mantiene completas las variables críticas, contiene únicamente nulos estructurales documentados y asigna exactamente una militancia a cada voto.

La estabilidad se asegura mediante ordenamiento determinista, funciones reutilizables, contratos de datos y ejecución automatizada de `F2_03_procesamiento_validacion.ipynb` y `F2_04_Integración.ipynb`. La trazabilidad se conserva mediante los datos raw, las tablas interim, la separación entre militancias observadas y analíticas, `reporte_calidad.csv` y `diagnostico_integracion.csv`. En consecuencia, el dataset final es íntegro, estable, verificable y apto para las etapas posteriores de modelamiento y análisis.

XI. Validación técnica y verificación del código

La estrategia de verificación contempla casos normales, casos límite y excepciones. Combina controles ejecutados dentro de los notebooks, manejo explícito de errores, pruebas automatizadas, contratos de datos, análisis estático e integración continua.

Cada control genera una evidencia verificable: salidas guardadas en los notebooks, matrices de validación, indicadores de aprobación, archivos de diagnóstico, resultados de Pytest, revisión de Ruff y registros de GitHub Actions.

Cobertura de los casos de prueba

Tabla 21

Cobertura de los casos de prueba

Tipo de caso	Verificación implementada	Evidencia
Casos normales	<code>F2_02_exploracion.ipynb</code> realiza un inventario de los 11 CSV disponibles en <code>F2/data</code> y selecciona las cinco tablas fuente utilizadas por el pipeline: proyecto, períodos, diputados, militancias y detalle nominal. Verifica dimensiones, tipos, nulos, duplicados y claves.	Perfil general, tablas de cardinalidad y matriz de controles guardados en <code>F2_02_exploracion.ipynb</code> .
Casos límite	Se controlan columnas completamente nulas, categorías únicas, campos opcionales, períodos abiertos, diputados sin votos y votaciones potencialmente unánimes. El EDA identifica 2 de 15 votaciones preliminarmente unánimes.	Salidas del EDA y observaciones registradas en <code>reporte_calidad.csv</code> .
Colecciones límite	Las pruebas contemplan militancias vacías, con un único elemento y con varios elementos, debido a que Zeep	<code>test_militancias_admite_coleccion_vacia</code> , <code>test_militancias_admite_un_solo_elemento</code> y <code>test_militancias_admite_varios_elementos</code> .

	puede representar estos casos con estructuras diferentes.	
Excepciones de entrada	Se verifica la existencia de archivos, columnas obligatorias, identificadores, fechas y esquemas. La ausencia del archivo genera FileNotFoundError; las columnas faltantes o valores inválidos generan ValueError.	Guardas explícitas en F2/src y funciones de normalización de F2_03_procesamiento_validacion.ipynb.
Excepciones de servicios	Las solicitudes utilizan timeout y raise_for_status(). El script 04 rechaza respuestas HTML de mantenimiento y contenido XML inválido.	descargar_xml_votaciones() y el parser utilizado por leer_xml_votacion().
Excepciones de integración	El proceso se detiene si se altera la cardinalidad, se duplican claves, falta un partido, no existe una militancia aplicable o más de una militancia coincide con el voto.	Aserciones y ValueError de F2_04_Integración.ipynb; RuntimeError cuando el veredicto final es rechazado.
Regresiones de datos	Los contratos comprueban claves, cardinalidad, completitud, intervalos temporales, ausencia de solapamientos y permanencia de excepciones documentadas.	Pruebas de F2/test/data_contracts/.

Resultados documentados

Los notebooks conservan los resultados intermedios y finales de cada etapa:

- F2_02_exploracion.ipynb ejecuta 16 controles exploratorios: 14 finalizan en OK y 2 en ADVERTENCIA.
- F2_03_procesamiento_validacion.ipynb produce 407 registros de militancia observada, 399 registros analíticos y 173 observaciones de calidad.
- F2_03_procesamiento_validacion.ipynb finaliza con F2_03_APROBADO=True.
- F2_04_Integración.ipynb genera una tabla de 1.993 filas y 34 columnas.
- Las 12 validaciones de integración finalizan en OK.
- F2_04_Integración.ipynb finaliza con F2_04_APROBADO=True.
- La tabla final no presenta duplicados en votacion_id + diputado_id, votos sin diputado, votos sin proyecto, partidos vacíos ni militancias ambiguas.

F2_04_Integración.ipynb calcula el veredicto a partir de la matriz de validaciones: si todos los controles están en estado OK, registra la aprobación; si alguno falla, muestra “Integración rechazada” y detiene la ejecución.

Pruebas automatizadas

La suite contiene 25 pruebas:

Tabla 22

Distribución de las pruebas automatizadas por grupo

Grupo	Cantidad	Cobertura
Pruebas unitarias	11	Funciones de extracción, transformación, escritura y casos límite de estructuras SOAP.
Contratos de datos	13	Claves, cardinalidad, completitud, intervalos, solapamientos, excepciones y reporte de calidad.
Prueba integral	1	Ejecución completa de F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb y comprobación de sus indicadores de aprobación.
Total	25	Cobertura funcional, estructural y de ejecución.

Las 24 pruebas rápidas se ejecutan sin consultar servicios externos. La prueba lenta abre y ejecuta los notebooks de procesamiento e integración mediante nbclient. La separación mediante el marcador slow permite obtener retroalimentación rápida sin renunciar a la verificación integral.

Ruff revisa F2/src y F2/test aplicando controles de errores, estilo, orden de importaciones y modernización de sintaxis. La revisión actual finaliza sin observaciones.

Integración continua

El workflow `.github/workflows/ci.yml` ejecuta tres trabajos independientes en cada pull request dirigido a main:

1. lint: instala el entorno bloqueado y ejecuta Ruff.
2. test-fast: ejecuta las 24 pruebas rápidas.
3. test-notebooks: instala el kernel y ejecuta la prueba integral de F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb.

La separación de trabajos permite identificar si una falla corresponde a calidad del código, funciones o contratos de datos, o ejecución integral de notebooks. Los registros de GitHub Actions conservan el resultado, duración y salida de cada trabajo asociado al pull request.

Cadena de trazabilidad

Tabla 23

Niveles de la cadena de trazabilidad del proyecto

Nivel	Evidencia conservada	Función dentro de la trazabilidad
Fuente	17 XML en F2/data/raw/	Preservan las respuestas originales.
Estructuración	Cinco CSV en F2/data/interim/	Permiten reconstruir la transformación del XML a tablas.
Exploración	Salidas y matriz de controles de F2_02_exploracion.ipynb	Documentan los problemas detectados antes de transformar.
Procesamiento	Seis productos de F2_03_procesamiento_validacion.ipynb y reporte_calidad.csv	Registran transformaciones, excepciones y reglas aplicadas.
Integración	big_table_analitica.csv y diagnostico_integracion.csv	Conservan el resultado y las métricas finales.
Pruebas	Archivos de F2/test/ y salidas de Pytest	Permiten repetir los controles de funciones y datos.
Automatización	Workflow y registros de GitHub Actions	Evidencian la ejecución de los controles en un entorno limpio.
Versionamiento	Commits y pull requests	Relacionan cada prueba o corrección con su autor y motivo.

Validaciones técnica en Notebook F1_Definición.ipynb

F1_Definición.ipynb contiene 21 celdas: 13 Markdown y 8 de código. Respeta el límite de la fase, ya que define y valida el proyecto sin consultar servicios web, descargar votaciones ni transformar los datos legislativos. Estas operaciones permanecen reservadas para F2.

Estructura del notebook

Tabla 24

Estructura del notebook y evidencia por sección

Sección	Contenido y evidencia
1–2	Contexto, problemática, preguntas de análisis y objetivos.
3	Alcance, exclusiones, supuestos y unidad de observación.

4	Configuración mediante ProjectConfig, una dataclass inmutable con método validate().
5	collect_environment() registra fecha UTC, Python, ejecutable, sistema operativo, pandas, NumPy, Jupyter, IPython, uv y Git.
6	Identificación de la raíz del repositorio, rama activa, último commit, remoto y estructura esperada, sin modificar el historial.
7	Plan estructurado de 13 etapas, vinculado con el mapa conceptual y operativo.
8	Ocho decisiones técnicas con fundamento y consecuencia verificable.
9	Seis validaciones automáticas mediante run_f1_checks() y una aserción final.
10	Síntesis de resultados e indicador de ejecución completa.
Referencias	Seis fuentes, incluido el artículo metodológico de B-Call.

Verificación de F1

El notebook comprueba:

1. Validez de la configuración.
2. Formato del boletín.
3. Declaración de un repositorio GitHub.
4. Orden correcto de las cuatro fases.
5. Existencia de las 13 etapas planificadas.
6. Numeración única y consecutiva.

La salida guardada muestra los seis resultados en True y la celda final imprime “F1 ejecutada de inicio a fin sin errores”. Su última modificación corresponde al commit b0a6c19 del 13-09-2026.

ProjectConfig.validate() también define el comportamiento frente a configuraciones inválidas: genera ValueError si el boletín no cumple el formato, si la URL no corresponde a GitHub o si se altera la secuencia F1–F4. Las consultas locales a herramientas utilizan manejo de FileNotFoundError y TimeoutExpired, dejando evidencia legible cuando un componente no está disponible.

XII. Notebook F1_Definición.ipynb (F1)

El notebook F1_Definición.ipynb materializa la implementación inicial del proyecto correspondiente a F1. Contiene 21 celdas —13 narrativas y 8 de código— y mantiene el alcance de la fase: define, configura y valida el proyecto, mientras que la obtención y transformación de los datos legislativos se reserva para F2.

El notebook no se limita a presentar antecedentes conceptuales. También implementa una configuración ejecutable, registra el entorno, verifica la conexión con el repositorio, representa el plan operativo como

datos estructurados y comprueba automáticamente la coherencia de los elementos definidos.

Contenido e implementación

Tabla 25

Contenido e implementación del notebook F1_Definición.ipynb

Sección	Contenido	Evidencia de implementación
1. Contexto y problemática	Presenta la disponibilidad de registros nominales y la necesidad de transformarlos en indicadores comparables e interpretables.	La problemática coincide con la desarrollada en el informe técnico.
2. Objetivos	Define el objetivo general y los objetivos específicos relacionados con obtención, procesamiento, integración, modelamiento y validación.	Los objetivos orientan las etapas F1–F4 representadas posteriormente en el plan.
3. Alcance, exclusiones y supuestos	Delimita la unidad de observación, universo, cobertura, fuentes, niveles de análisis y resultados esperados.	Tabla de siete dimensiones y supuestos que separan lo incluido en F1–F2 de lo proyectado para F3–F4.
4. Configuración central	Implementa ProjectConfig como dataclass inmutable mediante frozen=True y slots=True.	Centraliza boletín, repositorio, fases y rutas; validate() controla formato, URL y secuencia de fases.
5. Entorno reproducible	Implementa collect_environment().	Registra fecha UTC, Python, ejecutable, sistema operativo, pandas, NumPy, Jupyter, IPython, uv y Git.
6. Git y GitHub	Detecta la raíz del repositorio y consulta su estado sin modificarlo.	Registra rama activa, último commit, remoto configurado y estructura esperada.
7. Correspondencia con el mapa	Representa las 13 etapas del mapa y plan operativo como una estructura de datos.	La lista PHASE_PLAN se convierte en un DataFrame con fase, finalidad, acción, herramientas, entregable, commit y responsable.
8. Decisiones técnicas	Registra ocho decisiones iniciales junto con su fundamento y consecuencia.	Permite relacionar cada decisión con una evidencia verificable en F2 o en el repositorio.
9. Validaciones de F1	Implementa run_f1_checks().	Genera un reporte estructurado de seis controles y utiliza una aserción final para detener la ejecución si alguno falla.

10. Resultado inicial	Resume la configuración y el estado de la implementación.	La celda final informa si F1 se ejecutó completamente y quedó preparada para F2.
Referencias	Incorpora seis fuentes en formato APA.	Incluye documentación de Python, Jupyter, uv, GitHub, Cámara de Diputadas y Diputados y el artículo metodológico de B-Call.

Código base mínimo y flujo reproducible

El código de F1 ejemplifica el flujo reproducible definido en el mapa conceptual:

1. Se centralizan los parámetros en ProjectConfig.
2. Se valida que la configuración sea coherente.
3. Se registra el entorno efectivo de ejecución.
4. Se identifica el repositorio y su estado.
5. Se representa el plan de 13 etapas como datos estructurados.
6. Se documentan las decisiones técnicas iniciales.
7. Se ejecutan controles automáticos.
8. Se genera una síntesis final de estado.

Documentación técnica integrada

La documentación se encuentra incorporada en el propio notebook mediante:

- Celdas Markdown antes de cada bloque de código.
- Anotaciones de tipo y nombres descriptivos.
- Docstrings para las funciones principales.
- Mensajes explícitos frente a configuraciones inválidas.
- Tablas de alcance, planificación y decisiones.
- Registro del entorno y del contexto Git.
- Reportes estructurados de validación.
- Referencias metodológicas y técnicas.

Esta combinación permite comprender qué hace cada bloque, por qué se ejecuta y qué resultado se espera, sin depender de explicaciones externas al notebook.

Ejecución y validación

La ejecución guardada en el notebook presenta los seis controles de F1 en estado True:

Tabla 26*Resultados de los controles de la fase 1*

Control	Resultado
Configuración válida	True
Boletín con formato esperado	True
Repositorio GitHub declarado	True
Cuatro fases ordenadas	True
Trece etapas planificadas	True
Numeración única y consecutiva	True

Si alguno de estos controles falla, la aserción final detiene la ejecución. Al completarse correctamente, la celda de cierre imprime:

“F1 ejecutada de inicio a fin sin errores”.

La última modificación del notebook corresponde al commit b0a6c19, del 13-09-2026. Su historial también conserva la implementación inicial (0c7c64f) y su actualización metodológica (0e9a555), permitiendo reconstruir su evolución.

Coherencia y trazabilidad

La coherencia entre notebook, informe y repositorio se verifica mediante elementos comunes:

- Boletín 11092-07.
- Problemática, preguntas y objetivos.
- Alcance y unidad de observación.
- Secuencia F1–F4.
- Plan de 13 etapas.
- URL del repositorio GitHub.
- Separación entre notebooks de orquestación y módulos de ejecución.
- Uso de Python 3.12, uv y archivos de reproducibilidad.
- Validación previa al modelamiento.

El notebook obtiene automáticamente la rama, el commit y el remoto del repositorio, por lo que su ejecución queda vinculada al contexto versionado. A su vez, las decisiones definidas en F1 se materializan en F2 mediante la conservación de datos originales, la resolución temporal de militancias, la validación antes del modelamiento y la separación entre extracción, procesamiento e integración.

En consecuencia, F1_Definición.ipynb presenta de manera completa la implementación inicial: integra narrativa, código base funcional, configuración reproducible, documentación técnica, controles automáticos, ejecución sin errores y trazabilidad con el informe y GitHub.

XIII. Notebooks ejecutables de la Fase 2

La Fase 2 se implementa mediante cuatro notebooks ordenados de acuerdo con el flujo definido: **obtención** → **exploración** → **procesamiento y validación** → **integración**. Cada notebook comienza con una explicación de su finalidad, identifica las entradas utilizadas, describe las decisiones aplicadas y presenta resultados intermedios antes de generar sus salidas.

Los notebooks cumplen una función narrativa y de orquestación. La lógica reutilizable de extracción y validación se mantiene en F2/src, evitando concentrar todo el procesamiento en celdas secuenciales difíciles de probar.

Estructura del flujo

Tabla 27

Estructura del flujo de notebooks de la fase 2

Notebook	Rol dentro del flujo	Entradas	Resultados verificables
F2_01_obtencion.ipynb	Orquesta los cuatro módulos de extracción e incorpora la lógica mínima para relacionar las fechas de votación con el período legislativo correspondiente.	Boletín 11092-07 y servicios de la Cámara.	15 votaciones, 11 períodos, período 10 seleccionado, 157 diputados, 407 militancias y 1.993 votos nominales.
F2_02_exploracion.ipynb	Realiza un EDA de preparación sin modificar los datos originales.	Cinco tablas intermedias; la ejecución actual inventaría además los 11 CSV disponibles en F2/data.	Perfil de calidad, controles referenciales y temporales, conciliación de votos y matriz de 16 validaciones: 14 OK y 2 ADVERTENCIA.
F2_03_procesamiento_validacion.ipynb	Aplica las 21 reglas de normalización, limpieza, tratamiento	Cinco tablas de data/interim.	Seis productos procesados, 399 registros de militancia analítica y

	de nulos y resolución temporal acordadas por el equipo.		reporte_calidad.csv con 173 observaciones.
F2_04_Integración.ipynb	Integra proyecto, diputados y militancia vigente sobre el detalle nominal utilizado como tabla vertebral.	Cuatro tablas procesadas de F2_03_procesamiento_validacion.ipynb.	big_table_analitica.csv, con 1.993 filas y 34 columnas; diagnóstico y matriz de 12 validaciones en estado OK.

Orden y documentación de las celdas

Tabla 28

Orden y documentación de las celdas por notebook

Notebook	Celdas Markdown	Celdas de código	Forma de documentación
F2_01_obtencion.ipynb	7	12	Explica la finalidad de cada extracción, el servicio consultado y la revisión de los resultados generados.
F2_02_exploracion.ipynb	24	27	Introduce cada bloque de perfilamiento, integridad, temporalidad, conciliación y análisis exploratorio.
F2_03_procesamiento_validacion.ipynb	28	35	Documenta las 21 decisiones, sus fundamentos, reglas aplicadas y productos resultantes.
F2_04_Integración.ipynb	13	14	Explica la granularidad, el orden de los cruces, sus cardinalidades, las reglas temporales y el criterio de

			aprobación.
--	--	--	-------------

Resultados intermedios verificables

Los notebooks no presentan únicamente el resultado final. Cada etapa conserva evidencia que permite controlar el avance del flujo:

- F2_01_obtencion.ipynb muestra los DataFrames generados, las rutas de salida y los conteos obtenidos desde los servicios.
- F2_02_exploracion.ipynb presenta perfiles por tabla, distribuciones, claves, antiuniones, conciliaciones y la matriz exploratoria.
- F2_03_procesamiento_validacion.ipynb muestra los conteos antes y después del procesamiento, las anomalías detectadas, las reglas aplicadas y el contenido del reporte de calidad.
- F2_04_Integración.ipynb verifica la cardinalidad después de cada cruce, muestra los controles de militancia temporal y genera una matriz consolidada de validaciones.
- Los indicadores F2_03_APROBADO y F2_04_APROBADO permiten comprobar automáticamente que los notebooks alcanzaron correctamente su criterio de término.

La tabla final preserva las 1.993 observaciones del detalle nominal, mantiene la clave votacion_id + diputado_id sin duplicados y asigna exactamente un partido analítico a cada voto.

Ajuste respecto del plan original

El plan inicial contemplaba un único notebook de procesamiento y validación que coordinara las tareas de limpieza, integración y construcción de matrices. Durante el desarrollo, el equipo separó estas responsabilidades:

- F2_03_procesamiento_validacion.ipynb quedó encargado de normalizar y validar cada tabla de manera independiente.
- F2_04_Integración.ipynb quedó encargado de integrar las tablas, controlar las cardinalidades y construir la tabla analítica.

La modificación se adoptó porque el procesamiento por tabla y la integración cruzada presentan riesgos diferentes. F2_03_procesamiento_validacion.ipynb controla tipos, valores faltantes, categorías e intervalos; F2_04_Integración.ipynb controla claves, relaciones muchos-a-uno, vigencia temporal y preservación de filas.

La separación permite ejecutar nuevamente la integración sin repetir la obtención o limpieza, localizar errores con mayor precisión y probar F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb de manera independiente. Su impacto fue la incorporación de F2_04_Integración.ipynb, big_table_analitica.csv, diagnostico_integracion.csv y pruebas específicas para la integración.

Ejecución y verificación automática

Los cuatro notebooks no contienen salidas de error guardadas. Además, la ejecución de las etapas críticas de procesamiento e integración se verifica mediante F2/test/notebooks/test_notebooks_ejecutan.py, que:

1. Abre F2_03_procesamiento_validacion.ipynb y F2_04_Integración.ipynb.
2. Los ejecuta desde un kernel nuevo.
3. Detiene la prueba si alguna celda produce una excepción.
4. Comprueba la presencia de F2_03_APROBADO=True.
5. Comprueba la presencia de F2_04_APROBADO=True.

Esta prueba se ejecuta como trabajo independiente en GitHub Actions. Las pruebas rápidas validan adicionalmente las funciones y contratos de datos sin consultar los servicios externos.

F2_01_obtencion.ipynb depende de la disponibilidad de las API de la Cámara; por esta razón, sus funciones puras se prueban con datos simulados y sin red. F2_02_exploracion.ipynb opera sobre archivos versionados y conserva una ejecución secuencial completa con sus resultados exploratorios.

Trazabilidad con el repositorio y el informe

Tabla 29

Trazabilidad entre notebooks, repositorio e informe

Etapas	Evidencia en notebooks	Evidencia en el repositorio
Obtención	Conteos, DataFrames y rutas mostrados en F2_01_obtencion.ipynb.	XML en data/raw, CSV en data/interim y scripts 01–04.
Exploración	Perfiles, gráficos y matriz de validaciones en F2_02_exploracion.ipynb.	Datos intermedios y decisiones documentadas.
Procesamiento	Conteos, reglas y reporte mostrado en F2_03_procesamiento_validacion.ipynb.	Seis CSV procesados y reporte_calidad.csv.
Integración	Cruces, cardinalidades y controles mostrados en F2_04_Integración.ipynb.	big_table_analitica.csv y diagnostico_integracion.csv.
Pruebas	Indicadores de aprobación y matrices de control.	F2/test, workflow de CI, commits y pull requests.

Las cifras incluidas en el informe —15 votaciones, 157 diputados, 407 militancias observadas, 399 militancias analíticas y 1.993 votos— corresponden con los resultados guardados en los notebooks y los CSV versionados.

XIV. Vinculación del mapa conceptual con el avance de proyecto

El documento de Lucidchart contiene dos páginas complementarias:

1. **Mapa conceptual**, que representa las relaciones entre problema, datos, procesamiento, validación, modelamiento y comunicación.
2. **Plan operativo**, que organiza 13 etapas en F1–F4 e identifica finalidad, acción, herramientas, entregables, commit esperado y responsable.

La implementación conserva este flujo conceptual, pero actualiza el estado y la materialización técnica de cada etapa. La correspondencia se verifica mediante notebooks, scripts, datos, pruebas, archivos de configuración, commits y resultados versionados.

Correspondencia de las 13 etapas

Tabla 30

Correspondencia entre las 13 etapas planificadas y su estado actual

N.º	Etapla planificada	Estado actual	Materialización y evidencia
1	Definir la problemática — F1	Implementada	F1_Definición.ipynb, secciones 1–3; problemática, preguntas, objetivos y alcance; commits 0c7c64f y 0e9a555.
2	Configurar entorno — F1	Implementada	.python-version, pyproject.toml, uv.lock, uv.toml, estructura F1/F2 y GitHub; commits fc99d92 y 2160ea1.
3	Catalogar votaciones — F2	Implementada	01_extraer_votaciones_proyecto.py, F2_01_obtencion.ipynb, proyecto_ley.xml y proyecto_ley.csv; 15 votaciones.
4	Resolver períodos, diputados y militancias — F2	Implementada	Scripts 02 y 03; 11 períodos, período 10 seleccionado, 157 diputados y 407 militancias.
5	Extraer detalles de votos nominales — F2	Implementada	Script 04, 15 XML de votaciones y detalle_votaciones.csv con 1.993 registros.
6	Explorar los datos — F2	Implementada	F2_02_exploracion.ipynb; perfiles, controles referenciales, conciliación y matriz de 16 validaciones.
7	Procesar y normalizar — F2	Implementada con ajuste técnico	La lógica prevista para el script 05 se implementó en F2_03_procesamiento_validacion.ipynb mediante funciones de normalización; genera seis productos y 173 observaciones de calidad.
8	Integrar las fuentes — F2	Implementada con ajuste de diseño	La integración prevista para el script 06 se separó en F2_04_Integración.ipynb; genera big_table_analitica.csv y el diagnóstico de integración.
9	Validar y construir matrices — F2	Implementada con ajuste de alcance	La validación se distribuyó entre validaciones.py, matrices de control y F2/test. Se construyó la tabla longitudinal validada de 1.993×34 . La matriz diputado-votación se trasladó a F3 porque depende de la codificación del modelo.
10	Experimentar algoritmos — F3	Proyectada	No se han estimado posiciones ni ejecutado modelos. Utilizará la tabla analítica y construirá la matriz requerida por

			cada especificación.
11	Seleccionar modelos — F3	Proyectada	Se evaluarán estabilidad, sensibilidad, identificabilidad e interpretación antes de seleccionar el modelo.
12	Visualizar resultados — F4	Proyectada	Los gráficos exploratorios de F2 son controles de calidad y no sustituyen las visualizaciones sustantivas de F4.
13	Comunicar resultados — F4	Proyectada	Este informe documenta F1 y F2, pero no constituye la comunicación final de resultados políticos; F4_02_Comunicacion.ipynb aún no se ha creado.

Materialización de los elementos del mapa

Tabla 31

Materialización técnica de los elementos del mapa conceptual

Elemento conceptual	Planificación	Materialización técnica
Problemática y objetivos	Convertir votos nominales en evidencia reproducible sobre posición y cohesión.	Preguntas, objetivos, alcance y supuestos en F1 y en el informe.
Fuentes de datos	Servicios abiertos de la Cámara de Diputadas y Diputados.	Cuatro endpoints utilizados por los scripts 01–04 y 17 XML conservados en data/raw.
Herramientas	Python, Jupyter, pandas, Git, GitHub y gestión reproducible de dependencias.	Python 3.12, uv, pyproject.toml, uv.lock, notebooks, scripts, Pytest, Ruff y GitHub Actions.
Flujo reproducible	Obtención → exploración → procesamiento → integración → modelamiento → comunicación.	F2_01 → F2_02 → F2_03 → F2_04; F3 y F4 permanecen proyectadas.
Niveles de datos	Preservar origen, productos intermedios y resultados procesados.	data/raw, data/interim y data/processed, sin sobrescribir la evidencia original.
Decisiones técnicas	Conservar todas las votaciones, resolver militancias temporalmente, distinguir abstención de ausencia y validar antes de modelar.	Catálogo completo, militancias_observadas, militancias_analiticas, categorías originales y matrices de validación.
Organización del repositorio	Separar fases, notebooks, código, datos y documentación.	Carpetas F1/F2; subcarpetas notebooks, src, data, docs y test.
Estructura de notebooks	Notebooks narrativos que orquestan módulos reutilizables.	Celdas Markdown explicativas, scripts 01–04, validaciones.py y salidas verificables por etapa.
Control de	Validar claves, cobertura, temporalidad	Matriz exploratoria, 12 controles de

calidad	y consistencia antes del modelamiento.	integración, 25 pruebas, Ruff y CI.
Trazabilidad	Asociar actividades, entregables, commits y responsables.	Historial Git, pull requests, autores, mensajes semánticos, archivos generados y reportes de calidad.
Resultados intermedios	Entregar una base apta para experimentación.	big_table_analitica.csv, con 1.993 filas, 34 columnas y una militancia por voto.
Resultados sustantivos	Estimar posiciones, cohesión y estabilidad.	Proyectados para F3 y F4; no se presentan anticipadamente en esta entrega.

Ajustes entre la planificación y la implementación

Durante el desarrollo se realizaron tres ajustes explícitos:

Tabla 32

Ajustes realizados respecto de la planificación inicial

Ajuste	Justificación	Impacto
No crear los scripts 05–07 como archivos independientes	Las transformaciones requerían una explicación cercana a las decisiones y resultados intermedios.	El procesamiento quedó en F2_03_procesamiento_validacion.ipynb; las funciones transversales se trasladaron a validaciones.py y los controles repetibles a F2/test.
Crear F2_04_Integración.ipynb como notebook independiente	La integración entre tablas presenta riesgos de cardinalidad y temporalidad diferentes de la limpieza de cada tabla.	Permitió aislar los cruces, validar cada unión y ejecutar nuevamente la integración sin repetir el procesamiento.
Trasladar la matriz diputado-votación a F3	Su estructura depende de cómo se codifiquen abstenciones, ausencias, dispensas y votaciones unánimes para cada modelo con el que se experimentará.	F2 entrega una tabla longitudinal sin imponer prematuramente una codificación; F3 construirá las matrices específicas de experimentación.

Estos ajustes no modifican la finalidad general del mapa. Mantienen la secuencia conceptual y mejoran la separación de responsabilidades, la trazabilidad y la capacidad de prueba.

Estado consolidado

Las etapas 1 a 8 están completamente implementadas. La etapa 9 está materializada mediante la validación integral y la construcción de la tabla longitudinal; su componente de matriz para modelamiento fue trasladado explícitamente a F3. Las etapas 10 a 13 permanecen proyectadas.

De este modo, el avance técnico es coherente con el alcance de la entrega: F1 define y configura el proyecto; F2 obtiene, explora, procesa, integra y valida los datos; F3 realizará la experimentación y selección metodológica; y F4 visualizará y comunicará los resultados.

XV. Bibliografía y aspectos formales

Bibliografía técnica y formativa

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>

McKinney, W. (2022). *Python for data analysis: Data wrangling with pandas, NumPy, and Jupyter* (3.^a ed.). O'Reilly Media.

Documentación técnica oficial

Astral. (s. f.). *uv documentation*. Recuperado el 15 de septiembre de 2026, de <https://docs.astral.sh/uv/>

Conventional Commits. (s. f.). *Conventional Commits 1.0.0: Especificación para dar significado a los mensajes de los commits*. <https://www.conventionalcommits.org/es/v1.0.0/>

GitHub. (s. f.). *Pull requests*. Recuperado el 15 de septiembre de 2026, de <https://docs.github.com/en/pull-requests/reference/pull-requests>

Project Jupyter. (s. f.). *Project Jupyter documentation*. Recuperado el 15 de septiembre de 2026, de <https://docs.jupyter.org/en/latest/>

Python Software Foundation. (2026). *Python 3.12 documentation*. Recuperado el 15 de septiembre de 2026, de <https://docs.python.org/3.12/>

Fuente académica complementaria

Toro-Maureira, S., Reutter, J., Valenzuela, L., Alcatruz, D., & Valenzuela, M. (2025). B-Call: Integrating ideological position and voting cohesion in legislative behavior. *Frontiers in Political Science*, 7, Article 1670089. <https://doi.org/10.3389/fpos.2025.1670089>

Fuente primaria de datos

Cámara de Diputadas y Diputados de Chile. (s. f.). *Datos abiertos*. Recuperado el 15 de septiembre de 2026, de <https://www.camara.cl/transparencia/datosAbiertos.aspx>